

I/O streams -File Streams-Applets-Applet Life Cycle - String Buffer-Char Array-Java Utility classes-Calendar-Date-Random-Scanner-Timer-Vector

Java IO Stream

Java performs I/O through **Streams**. A Stream is linked to a physical layer by java I/O system to make input and output operation in java. In general, a stream means continuous flow of data. Streams are clean way to deal with input/output without having every part of your code understand the physical.

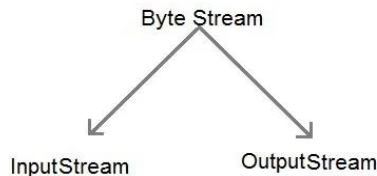
Java encapsulates Stream under **java.io** package. Java defines two types of streams. They are,

Byte Stream : It provides a convenient means for handling input and output of byte.

Character Stream : It provides a convenient means for handling input and output of characters. Character stream uses Unicode and therefore can be internationalized.

Java Byte Stream Classes

Byte stream is defined by using two abstract class at the top of hierarchy, they are `InputStream` and `OutputStream`.



These two abstract classes have several concrete classes that handle various devices such as disk files, network connection etc

Some important Byte stream classes.

Stream class	Description
BufferedInputStream	Used for Buffered Input Stream.
BufferedOutputStream	Used for Buffered Output Stream.
DataInputStream	Contains method for reading java standard datatype
DataOutputStream	An output stream that contain method for writing java standard data type
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that write to a file.
InputStream	Abstract class that describe stream input.
OutputStream	Abstract class that describe stream output.
PrintStream	Output Stream that contain <code>print()</code> and <code>println()</code> method

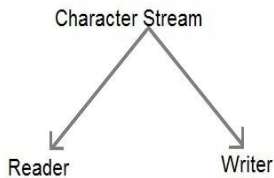
These classes define several key methods. Two most important are

read() : reads byte of data.

write() : Writes byte of data.

Java Character Stream Classes

Character stream is also defined by using two abstract class at the top of hierarchy, they are Reader and Writer.



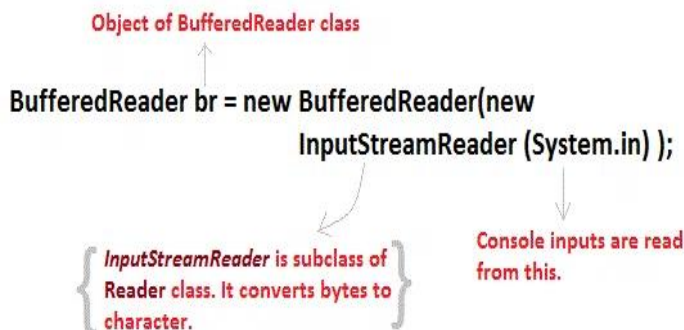
These two abstract classes have several concrete classes that handle unicode character.

Some important Character stream classes

Stream class	Description
BufferedReader	Handles buffered input stream.
BufferedWriter	Handles buffered output stream.
FileReader	Input stream that reads from file.
FileWriter	Output stream that writes to file.
InputStreamReader	Input stream that translate byte to character
OutputStreamReader	Output stream that translate character to byte.
PrintWriter	Output Stream that contain <code>print()</code> and <code>println()</code> method.
Reader	Abstract class that define character stream input
Writer	Abstract class that define character stream output

Reading Console Input

We use the object of BufferedReader class to take inputs from the keyboard.



Reading Characters

`read()` method is used with `BufferedReader` object to read characters. As this function returns integer type value has we need to use typecasting to convert it into **char** type.

int read() throws IOException

Below is a simple example explaining character input.

```
class CharRead
{
    public static void main( String args[])
```

```

{
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    char c = (char)br.read(); //Reading character
}
}

```

Reading Strings in Java

To read string we have to use readLine() function with BufferedReader class's object.

String readLine() throws IOException

Program to take String input from Keyboard in Java

```

import java.io.*;
class MyInput
{
    public static void main(String[] args)
    {
        String text;
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);
        text = br.readLine(); //Reading String
        System.out.println(text);
    }
}

```

Program to read from a file using BufferedReader class

```

import java. Io *;
class ReadTest
{
    public static void main(String[] args)
    {
        try
        {
            File fl = new File("d:/myfile.txt");
            BufferedReader br = new BufferedReader(new FileReader(fl)) ;
            String str;
            while ((str=br.readLine())!=null)
            {
                System.out.println(str);
            }
            br.close();
            fl.close();
        }
        catch(IOException e) {
            e.printStackTrace();
        }
    }
}

```

Program to write to a File using FileWriter class

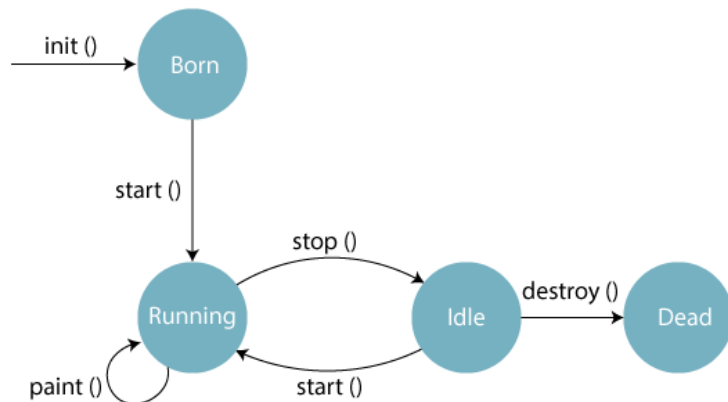
```
import java. Io *;  
class WriteTest  
{  
    public static void main(String[] args)  
    {  
        try  
        {  
            File fl = new File("d:/myfile.txt");  
            String str="Write this string to my file";  
            FileWriter fw = new FileWriter(fl) ;  
            fw.write(str);  
            fw.close();  
            fl.close();  
        }  
        catch (IOException e)  
        { e.printStackTrace(); }  
    }  
}
```

Applet Life Cycle in Java

In Java, an applet is a special type of program embedded in the web page to generate dynamic content. Applet is a class in Java.

The applet life cycle can be defined as the process of how the object is created, started, stopped, and destroyed during the entire execution of its application. It basically has five core methods namely init(), start(), stop(), paint() and destroy(). These methods are invoked by the browser to execute.

Along with the browser, the applet also works on the client side, thus having less processing time. Methods of Applet Life Cycle



There are five methods of an applet life cycle, and they are:

init(): The init() method is the first method to run that initializes the applet. It can be invoked only once at the time of initialization. The web browser creates the initialized objects, i.e., the web browser (after checking the security settings) runs the init() method within the applet.

start(): The start() method contains the actual code of the applet and starts the applet. It is invoked immediately after the init() method is invoked. Every time the browser is loaded or refreshed, the start() method is invoked. It is also invoked whenever the applet is maximized, restored, or moving from one tab to another in the browser. It is in an inactive state until the init() method is invoked.

stop(): The stop() method stops the execution of the applet. The stop () method is invoked whenever the applet is stopped, minimized, or moving from one tab to another in the browser, the stop() method is invoked. When we go back to that page, the start() method is invoked again.

destroy(): The destroy() method destroys the applet after its work is done. It is invoked when the applet

window is closed or when the tab containing the webpage is closed. It removes the applet object from memory and is executed only once. We cannot start the applet once it is destroyed.

paint(): The paint() method belongs to the Graphics class in Java. It is used to draw shapes like circle, square, trapezium, etc., in the applet. It is executed after the start() method and when the browser or applet windows are resized.

Sequence of method execution when an applet is executed:

init()
start()
paint()

Sequence of method execution when an applet is executed:

stop()
destroy()

Applet Life Cycle Working

The Java plug-in software is responsible for managing the life cycle of an applet.

An applet is a Java application executed in any web browser and works on the client-side. It doesn't have the main() method because it runs in the browser. It is thus created to be placed on an HTML page.

The init(), start(), stop() and destroy() methods belongs to the **applet.Applet** class.

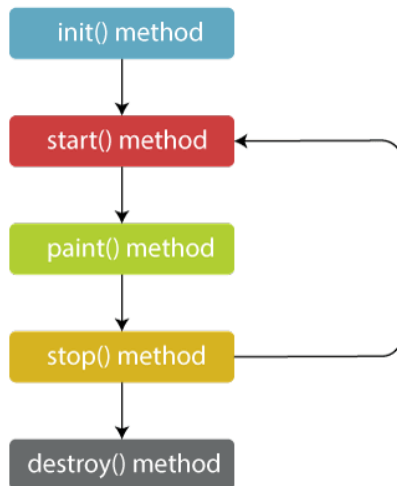
The paint() method belongs to the **awt.Component** class.

In Java, if we want to make a class an Applet class, we need to extend the **Applet**

Whenever we create an applet, we are creating the instance of the existing Applet class. And thus, we can use all the methods of that class.

Flow of Applet Life Cycle:

These methods are invoked by the browser automatically. There is no need to call them explicitly.



Syntax of entire Applet Life Cycle in Java

```
class TestAppletLifeCycle extends Applet {  
    public void init() {  
        // initialized objects  
    }  
    public void start() {  
        // code to start the applet  
    }  
    public void paint(Graphics graphics) {  
        // draw the shapes  
    }  
    public void stop() {  
        // code to stop the applet  
    }  
}
```

```
public void destroy() {  
    // code to destroy the applet  
}  
}
```

Constructor Of Applet Class

The Applet class is just like any other class as an Applet Constructor is simply the subclass constructor of the Applet class. Therefore, Because the applet constructor is just like or any other constructor, it cannot be overridden, so Constructors perform any Necessary initialization for the new object or Create the new Object for it.

Applet() – It Constructs a new Applet.

Example of Applet: The Applet program using appletviewer-

// This is a Simple Java Applet

// program using appletviewer

```
import java.applet.*;  
import java.awt.*;  
  
/*  
<applet code="AppletExp1" width=600 height=300>  
</applet>  
*/  
  
public class AppletExp1 extends Applet {  
    public void init()  
    {  
        System.out.println("Initializing an applet");  
    }  
  
    public void start()  
    {  
        System.out.println("Starting an applet");  
    }  
    public void stop()  
  
    {  
        System.out.println("Stopping an applet");  
    }  
    public void destroy()  
  
    {  
        System.out.println("Destroying an applet");  
    }  
}
```

By using Appletviewer, type the following command at the command prompt-

```
appletviewer AppletExp1.java
```

Output:

Then after An Window is Opening the Final applet Output Applet Window-



Advantages of Applet

It runs inside the browser and works on the Client-side, so it takes less time to respond.

It is more Secured

It can be Executed By multi-platforms with any Browsers, i.e., Windows, Mac Os, Linux Os.

Disadvantages of Applet

A plugin is required at the client browser(User Side) to execute an Applet.

Displaying Graphics in Applet

java.awt.Graphics class provides many methods for graphics programming.

Commonly used methods of Graphics class:

public abstract void drawString(String str, int x, int y): is used to draw the specified string.

public void drawRect(int x, int y, int width, int height): draws a rectangle with the specified width and height.

public abstract void fillRect(int x, int y, int width, int height): is used to fill rectangle with the default color and specified width and height.

public abstract void drawOval(int x, int y, int width, int height): is used to draw oval with the specified width and height.

public abstract void fillOval(int x, int y, int width, int height): is used to fill oval with the default color and specified width and height.

public abstract void drawLine(int x1, int y1, int x2, int y2): is used to draw line between the points(x1, y1) and (x2, y2).

public abstract boolean drawImage(Image img, int x, int y, ImageObserver observer): is used draw the specified image.

public abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle): is used draw a circular or elliptical arc.

public abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle): is used to fill a circular or elliptical arc.

public abstract void setColor(Color c): is used to set the graphics current color to the specified color.

public abstract void setFont(Font font): is used to set the graphics current font to the specified font.

INCORPORATING GRAPHICS

```
import java.applet.*;import
```

```
java.awt.*;
```

```
/*<applet code=UsingGraphics width=600 height=500></applet>*/public class UsingGraphics
```

```
extends Applet
```

```
{
```

```
public void paint(Graphics g)
```

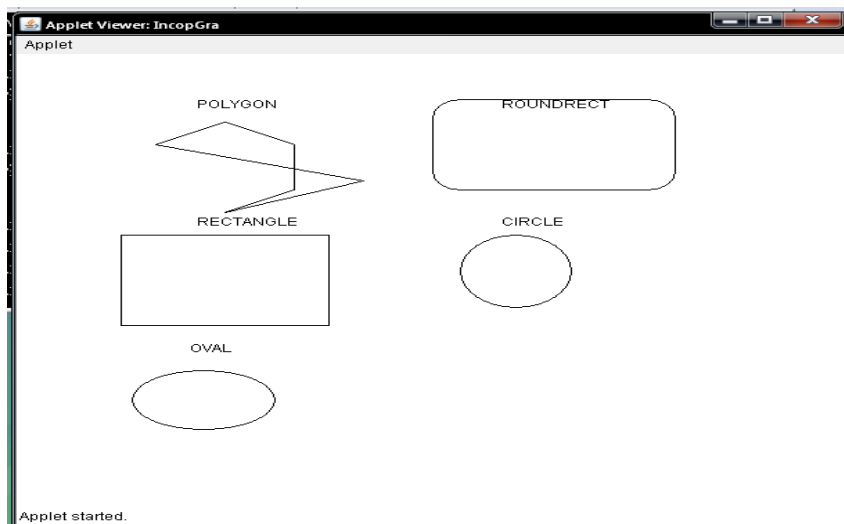
```

{
g.drawOval(83,350,103,65); g.drawRect(75,200,150,100);
g.drawRoundRect(300,50,175,100,40,40);g.drawOval(320,200,80,80);
g.drawString("POLYGON",130,60);
g.drawString("ROUNDRECT",350,60);
g.drawString("RECTANGLE",130,190);
g.drawString("CIRCLE",350,190);
g.drawString("OVAL",125,330); int
a[]={100,150,200,200,150,250};
int b[]={100,75,100,150,175,140};
g.drawPolygon(a,b,6);
}
}

```

Compile: javac UsingGraphics.java **Execute:**
 appletviewer UsingGraphics.java

OUTPUT



WORKING WITH COLORS AND FONTS

```

import java.applet.Applet;import
java.awt.Font; import java.awt.Color;
import java.awt.Graphics;
/* <APPLET CODE ="usingfont.class" WIDTH=300 HEIGHT=200> </APPLET> */public
class usingfont extends Applet

```



```

{
Font f; String m;
    public void init()
    {
        f=new Font("Arial",Font.ITALIC,20);
        m="Welcome to Java";
        setFont(f);
    }
    public void paint(Graphics g)
    {
        Color c=new Color(0,255,0);
        g.setColor(c); g.drawString(m,4,20);
    }
}
} //class ends

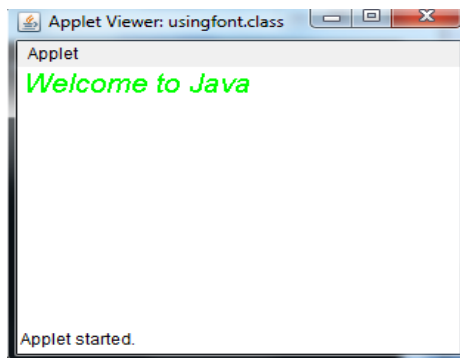
```

```

D:\jdk1.6.0\bin>javac usingfont.java
D:\jdk1.6.0\bin>appletviewer usingfont.java

```

OUTPUT



Character Array in Java

In Java, a character array is a data structure used to store a sequence of characters. The characters are stored in contiguous memory locations and can be accessed by their index, similar to an array of integers or any other data type.

Declaring a Character Array

A character array can be declared in the following way:

```
char[] charArray;
```

This declares a reference variable called charArray that can store the memory address of a character array. To initialize the array and allocate memory for it, you can use the new keyword:

```
charArray = new char[10];
```

This creates an array with a length of 10, which means it can store 10 characters. The elements of the array are initialized to the default value of the char data type, which is the null character ('      ').

Alternatively, you can declare and initialize a character array in a single line:

```
char[] charArray = new char[10];
```

You can also initialize a character array with a set of characters by using the curly braces {} notation:

```
char[] charArray = {'a', 'b', 'c', 'd', 'e'};
```

This creates an array with a length of 5 and assigns the characters 'a', 'b', 'c', 'd', and 'e' to the elements at indexes 0 through 4, respectively.

Accessing Elements in a Character Array

You can access the elements of a character array using the array name followed by the element's index in square brackets. For example:

```
charArray[0] = 'f';
```

This assigns the character 'f' to the element at index 0 of the charArray. You can also use the array elements to perform operations. For example:

```
System.out.println(charArray[0] + " " + charArray[1]);
```

This prints the characters at index 0 and 1 of the charArray, separated by a space.

Iterating Over a Character Array

You can use a for loop to iterate over the elements of a character array. For example:

Java

```
for (int i = 0; i < charArray.length; i++) {  
    System.out.println(charArray[i]);  
}
```

This prints each element of the charArray on a new line.

Another way to iterate over a character array is to use the enhanced for loop

(also known as the for-each loop):

Java

```
for (char c : charArray) {  
    System.out.println(c);  
}
```

This also prints each element of the charArray on a new line.

Comparing Character Arrays

To compare two character arrays for equality, you can use the Arrays.equals() method:

Java

```
char[] charArray1 = {'a', 'b', 'c', 'd', 'e'};  
char[] charArray2 = {'a', 'b', 'c', 'd', 'e'};  
  
if (Arrays.equals(charArray1, charArray2)) {  
    System.out.println("The arrays are equal.");  
} else {  
    System.out.println("The arrays are not equal.");  
}
```

This compares the elements of charArray1 and charArray2, and if all the elements are identical, it prints "The arrays are equal." Otherwise, it prints "The arrays are not equal."

Converting a String to a Character Array

You can convert a String to a character array using the toCharArray() method:

Java

```
String str = "Hello World";  
char[] charArray = str.toCharArray();
```

This creates a character array with the same characters as the string and assigns it to the charArray variable.

Converting a Character Array to a String

You can convert a character array to a String using the String constructor that takes a character array as an argument:

Java

```
char[] charArray = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd'};  
String str = new String(charArray);
```

This creates a new String object with the same characters as the charArray, and assigns it to the str variable.

Character Array in Java

In Java, a character array is a data structure used to store a sequence of characters. The characters are stored in contiguous memory locations and can be accessed by their index, similar to an array of integers or any other data type.

Declaring a Character Array

A character array can be declared in the following way:

```
char[] charArray;
```

This declares a reference variable called charArray that can store the memory address of a character array.

To initialize the array and allocate memory for it, you can use the new keyword:

```
charArray = new char[10];
```

This creates an array with a length of 10, which means it can store 10 characters. The elements of the array are initialized to the default value of the char data type, which is the null character ('\u0000').

Alternatively, you can declare and initialize a character array in a single line:

```
char[] charArray = new char[10];
```

You can also initialize a character array with a set of characters by using the curly braces {} notation:

```
char[] charArray = {'a', 'b', 'c', 'd', 'e'};
```

This creates an array with a length of 5 and assigns the characters 'a', 'b', 'c', 'd', and 'e' to the elements at indexes 0 through 4, respectively.

Accessing Elements in a Character Array

You can access the elements of a character array using the array name followed by the element's index in square brackets. For example:

```
charArray[0] = 'f';
```

This assigns the character 'f' to the element at index 0 of the charArray. You can also use the array elements to perform operations. For example:

```
System.out.println(charArray[0] + " " + charArray[1]);
```

This prints the characters at index 0 and 1 of the charArray, separated by a space.

Iterating Over a Character Array

You can use a for loop to iterate over the elements of a character array. For example:

Java

```
for (int i = 0; i < charArray.length; i++) {  
    System.out.println(charArray[i]);  
}
```

This prints each element of the charArray on a new line.

Another way to iterate over a character array is to use the enhanced for loop (also known as the for-each loop):

Java

```
for (char c : charArray) {  
    System.out.println(c);  
}
```

This also prints each element of the charArray on a new line.

Comparing Character Arrays

To compare two character arrays for equality, you can use the Arrays.equals() method:

Java

```
char[] charArray1 = {'a', 'b', 'c', 'd', 'e'};  
char[] charArray2 = {'a', 'b', 'c', 'd', 'e'};  
  
if (Arrays.equals(charArray1, charArray2)) {  
    System.out.println("The arrays are equal.");  
} else {  
    System.out.println("The arrays are not equal.");  
}
```

This compares the elements of charArray1 and charArray2, and if all the elements are identical, it prints “The arrays are equal.” Otherwise, it prints “The arrays are not equal.”

Converting a String to a Character Array

You can convert a String to a character array using the toCharArray() method:

Java

```
String str = "Hello World";  
char[] charArray = str.toCharArray();
```

This creates a character array with the same characters as the string and assigns it to the charArray variable.

Converting a Character Array to a String

You can convert a character array to a String using the String constructor that takes a character array as an argument:

Java

```
char[] charArray = {'H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd'};  
String str = new String(charArray);
```

This creates a new String object with the same characters as the charArray, and assigns it to the str variable.

Difference between String and Character array in Java

Unlike C/C++ Character arrays and Strings are two different things in Java. Both Character Arrays and Strings are a collection of characters but are different in terms of properties.

Differences between Strings and Character Arrays:

Property	String	Character Array
Definition	A sequence of characters is represented as a single data type.	A sequential collection of characters of the data type char.

Property	String	Character Array
Mutability	Immutable – Once created, their values cannot be changed.	Mutable – Individual characters can be modified.
Built-in Functions	Provides many built-in functions like substring(), charAt(), length(), etc.	No built-in functions for operations.
Concatenation	Strings can be concatenated using the + operator.	Cannot use + for concatenation; must use loops or utility methods.
Access Characters	Characters can be accessed using the charAt() method.	Characters can be accessed using array indexing ([]).
Memory Storage	Can be stored anywhere in memory, typically managed by the JVM.	Elements are stored contiguously in increasing memory locations.
String Constant Pool	Strings are stored in String Constant Pool if they are string literals or interned, otherwise they are stored in the heap.	Stored in the Heap.
Security	Not preferred for storing passwords due to immutability and persistence in the String Pool.	Preferred for storing passwords as they can be explicitly cleared after use.
Conversion	Can be converted to a Character Array using the toCharArray() method. Example: String s = "GEEKS"; char[] ch = s.toCharArray();	Can be converted to a String using the String constructor. Example: char[] a = {'G', 'E', 'E', 'K', 'S'}; String A = new String(a);

Key Differences:

Immutability vs. Mutability:

String: Immutable, meaning once a String object is created, it cannot be altered. This property makes them suitable for use as keys in hashmaps or for string manipulation without side effects.

Character Array: Mutable, allowing changes to individual characters, making them more suitable for scenarios where frequent modifications are required, like password storage.

Built-in Functions:

String: Java provides a rich set of built-in methods for various operations, including searching, splitting, and replacing substrings.

Character Array: Lacks built-in methods, so manual handling or utility methods are required for similar operations.

Memory Management:

String: Stored in the String Constant Pool when it is defined as literal, in double quotes – "Hello, World!". This allows for reuse of common strings, thus saving memory.

If it is created by using new keyword via constructors **new String("Hello, World!")** they are stored in normal

heap area.

Character Array: Stored in the heap, providing direct and contiguous access to memory locations.

Security Considerations:

String: Due to immutability and persistence in the String Pool, they can be a security risk for sensitive data like passwords.

Character Array: Since they can be explicitly modified and cleared, they are preferred for storing sensitive information.

What is Utility Class?

The utility class is helper. Let's say we have a little logic in the code and we use this logic throughout the project, instead of writing this logic over and over in each class, we can create a new class and add this logic to the newly created class. And then we can call this logic wherever we want. Maybe we have 10 lines of code in our logic, we can replace this 10 lines of code with 1 line.

Why we need Utility Class?

Utility class helps us to improve code quality, readability. Also you can use Utility class to write good clean code. Other developers can understand your code easily. This is the main idea to use this class everywhere in the project. Now let's see this Utility class in action.

Java.util.Random class in Java

Random class is used to generate pseudo-random numbers in java. An instance of this class is thread-safe. The instance of this class is however cryptographically insecure. This class provides various method calls to generate different random data types such as float, double, int.

Constructors:

Random(): Creates a new random number generator

Random(long seed): Creates a new random number generator using a single long seed

Declaration:

```
public class Random
    extends Object
    implements Serializable
```

Methods:

java.util.Random.doubles(): Returns an effectively unlimited stream of pseudo random double values, each between zero (inclusive) and one (exclusive)

Syntax:

```
public DoubleStream doubles()
```

Returns: a stream of pseudorandom double values

java.util.Random.ints(): Returns an effectively unlimited stream of pseudo random int values

Syntax:

```
public IntStream ints()
```

Returns: a stream of pseudorandom int values

java.util.Random.longs(): Returns an effectively unlimited stream of pseudo random long values

Syntax:

```
public LongStream longs()
```

Returns: a stream of pseudorandom long values

next(int bits): java.util.Random.next(int bits) Generates the next pseudo random number

Syntax:

```
protected int next(int bits)
```

Parameters: bits - random bits

Returns: the next pseudo random value from this random number generator's sequence

java.util.Random.nextBoolean(): Returns the next pseudo random, uniformly distributed boolean value from this random number generator's sequence

Syntax:

```
public boolean nextBoolean()
```

Returns: the next pseudorandom, uniformly distributed boolean value from this random number generator's sequence

java.util.Random.nextBytes(byte[] bytes) :Generates random bytes and places them into a user-supplied byte array

Syntax:

```
public void nextBytes(byte[] bytes)
```

Parameters: bytes - the byte array to fill with random bytes

Throws:

NullPointerException - if the byte array is null

java.util.Random.nextDouble(): Returns the next pseudo random, uniformly distributed double value between 0.0 and 1.0 from this random number generator's sequence

Syntax:

```
public double nextDouble()
```

Returns: the next pseudo random, uniformly distributed double value between 0.0 and 1.0 from this random number generator's sequence

java.util.Random.nextFloat(): Returns the next pseudo random, uniformly distributed float value between 0.0 and 1.0 from this random number generator's sequence

Syntax:

```
public float nextFloat()
```

Returns: the next pseudorandom, uniformly distributed float value between 0.0 and 1.0 from this random number generator's sequence

java.util.Random.nextGaussian(): Returns the next pseudo random, Gaussian ("normally") distributed double value with mean 0.0 and standard deviation 1.0 from this random number generator's sequence

Syntax:

```
public double nextGaussian()
```

Returns: the next pseudorandom, Gaussian ("normally") distributed double value with mean 0.0 and standard deviation 1.0 from this random number generator's sequence

java.util.Random.nextInt(): Returns the next pseudorandom, uniformly distributed int value from this random number generator's sequence

Syntax:

```
public int nextInt()
```

Returns:

the next pseudorandom, uniformly distributed int value from this random number generator's sequence

java.util.Random.nextInt(int bound): Returns a pseudo random, uniformly distributed int value between 0 (inclusive) and the specified value (exclusive), drawn from this random number generator's sequence

Syntax:

```
public int nextInt(int bound)
```

Parameters:

bound - the upper bound (exclusive). Must be positive.

Returns:

the next pseudorandom, uniformly distributed int value between zero (inclusive) and bound (exclusive) from this random number generator's sequence

Throws:

IllegalArgumentException - if bound is not positive

java.util.Random.nextLong(): Returns the next pseudorandom, uniformly distributed long value from this random number generator's sequence

Syntax:

```
public long nextLong()
```

Returns:

the next pseudorandom, uniformly distributed long value from this random number generator's sequence
java.util.Random.setSeed(long seed): Sets the seed of this random number generator using a single long seed

Syntax:

```
public void setSeed(long seed)
```

Parameters:

seed - the initial seed

```
import java.util.Random;
```

```
public class generateRandom{
```

```
    public static void main(String args[])
    {
        // create instance of Random class
        Random rand = new Random();

        // Generate random integers in range 0 to 999
        int rand_int1 = rand.nextInt(1000);
        int rand_int2 = rand.nextInt(1000);

        // Print random integers
        System.out.println("Random Integers: "+rand_int1);
        System.out.println("Random Integers: "+rand_int2);

        // Generate Random doubles
        double rand_dub1 = rand.nextDouble();
        double rand_dub2 = rand.nextDouble();

        // Print random doubles
        System.out.println("Random Doubles: "+rand_dub1);
        System.out.println("Random Doubles: "+rand_dub2);
    }
}
```

Output

Random Integers: 618

Random Integers: 877

Random Doubles: 0.11981638980670772

Random Doubles: 0.7288425427367139

What is the Calendar Class in Java?

The Calendar class in Java is a fundamental component of the java.util package, designed to facilitate time-related operations in a robust and versatile manner. It provides a broad range of functionalities for handling dates, times, and time zones. Java developers often rely on this class to streamline tasks related to scheduling, event handling, and other temporal operations.

The Calendar class in Java is an abstract class that provides methods for working with dates and times. It allows for the conversion between a specific instant in time and a set of calendar fields such as MONTH, YEAR, HOUR, among others. Being an abstract class, it cannot be instantiated directly; instead, one must use the Calendar.getInstance() method to get a Calendar object set to the current date and time in the default time

zone and locale.

Key Methods of the Calendar Class

The Calendar class includes a variety of methods that facilitate the manipulation and interpretation of date and time fields. Here are some of the most commonly used methods:

`add(int field, int amount)`: Adds or subtracts the specified amount of time to the given calendar field, based on the calendar's rules.

`get(int field)`: Returns the value of the given calendar field.

`getMaximum(int field)`: Returns the maximum value for the given calendar field of this Calendar instance.

`getMinimum(int field)`: Returns the minimum value for the given calendar field of this Calendar instance.

`getTime()`: Returns a Date object representing this Calendar's time value.

`set(int field, int value)`: Sets the given calendar field to the specified value.

`getTimeInMillis()`: Returns the current time in milliseconds.

Practical Examples

Here are some examples demonstrating the use of the Calendar class:

```
import java.util.Calendar;
```

```
public class CalendarExample {  
    public static void main(String[] args) {  
        // Get the current date and time  
        Calendar calendar = Calendar.getInstance();  
        System.out.println("The current date is: " + calendar.getTime());
```

```
        // Subtract 15 days from the current date  
        calendar.add(Calendar.DATE, -15);  
        System.out.println("15 days ago: " + calendar.getTime());
```

```
        // Add 4 months to the current date  
        calendar.add(Calendar.MONTH, 4);  
        System.out.println("4 months later: " + calendar.getTime());
```

```
        // Add 2 years to the current date  
        calendar.add(Calendar.YEAR, 2);  
        System.out.println("2 years later: " + calendar.getTime());  
    }  
}
```

In the above example, the `getInstance()` method is used to obtain a Calendar object. The `add()` method is then used to manipulate the date by subtracting days and adding months and years. The `getTime()` method is used to print the modified dates.

```
import java.util.*;  
public class CalendarExample4 {  
    public static void main(String[] args) {  
        Calendar calendar = Calendar.getInstance();  
        int maximum = calendar.getMaximum(Calendar.DAY_OF_WEEK);  
        System.out.println("Maximum number of days in week: " + maximum);  
        maximum = calendar.getMaximum(Calendar.WEEK_OF_YEAR);  
        System.out.println("Maximum number of weeks in year: " + maximum);  
    }  
}
```

Maximum number of days in week: 7
Maximum number of weeks in year: 53

```
import java.util.*;
public class CalendarExample2{
    public static void main(String[] args) {
        Calendar calendar = Calendar.getInstance();
        System.out.println("At present Calendar's Year: " + calendar.get(Calendar.YEAR));
        System.out.println("At present Calendar's Day: " + calendar.get(Calendar.DATE));
    }
}
```

At present Calendar's Year: 2024
At present Calendar's Day: 1

```
import java.util.Calendar;
public class CalendarExample1 {
    public static void main(String[] args) {
        Calendar calendar = Calendar.getInstance();
        System.out.println("The current date is : " + calendar.getTime());
        calendar.add(Calendar.DATE, -15);
        System.out.println("15 days ago: " + calendar.getTime());
        calendar.add(Calendar.MONTH, 4);
        System.out.println("4 months later: " + calendar.getTime());
        calendar.add(Calendar.YEAR, 2);
        System.out.println("2 years later: " + calendar.getTime());
    }
}
```

The current date is : Tue Oct 01 04:30:52 UTC 2024
15 days ago: Mon Sep 16 04:30:52 UTC 2024
4 months later: Thu Jan 16 04:30:52 UTC 2025
2 years later: Sat Jan 16 04:30:52 UTC 2027

Java User Input

The Scanner class is used to get user input, and it is found in the java.util package.

To use the Scanner class, create an object of the class and use any of the available methods found in the Scanner class documentation. In our example, we will use the `nextLine()` method, which is used to read Strings:

Example

```
import java.util.Scanner; // Import the Scanner class
```

```
class Main {
    public static void main(String[] args) {
        Scanner myObj = new Scanner(System.in); // Create a Scanner object
        System.out.println("Enter username");

        String userName = myObj.nextLine(); // Read user input
        System.out.println("Username is: " + userName); // Output user input
    }
}
```

Enter username

vels

Username is: vels

Input Types

In the example above, we used the `nextLine()` method, which is used to read Strings. To read other types, look at the table below:

Method	Description
<code>nextBoolean()</code>	Reads a boolean value from the user
<code>nextByte()</code>	Reads a byte value from the user
<code>nextDouble()</code>	Reads a double value from the user
<code>nextFloat()</code>	Reads a float value from the user
<code>nextInt()</code>	Reads a int value from the user
<code>nextLine()</code>	Reads a String value from the user
<code>nextLong()</code>	Reads a long value from the user
<code>nextShort()</code>	Reads a short value from the user

```
import java.util.Scanner;
```

```
class Main {  
    public static void main(String[] args) {  
        Scanner myObj = new Scanner(System.in);  
  
        System.out.println("Enter name, age and salary:");  
  
        // String input  
        String name = myObj.nextLine();  
  
        // Numerical input  
        int age = myObj.nextInt();  
        double salary = myObj.nextDouble();  
  
        // Output input by user  
        System.out.println("Name: " + name);  
        System.out.println("Age: " + age);  
        System.out.println("Salary: " + salary);  
    }  
}
```

```
Enter name, age and salary:
```

```
Latha
```

```
35
```

```
50000
```

```
Name: Latha
```

```
Age: 35
```

```
Salary: 50000
```

Timer Class:

The **Java Timer** class provides facility for threads to schedule tasks for future execution in a background thread.

This class is thread-safe i.e multiple threads can share a single Timer object without the need for external synchronization.

This class schedules tasks for one-time execution, or for repeated execution at regular intervals.

All constructors start a timer thread.

Class declaration

Following is the declaration for **java.util.Timer** class –

```
public class Timer  
    extends Object
```

Learn **Java** in-depth with real-world projects through our **Java certification course**. Enroll and become a certified expert to boost your career.

Class constructors

Sr.No.	Constructor & Description
1	Timer() This constructor creates a new timer.
2	Timer(boolean isDaemon) This constructor creates a new timer whose associated thread may be specified to run as a daemon.
3	Timer(String name) This constructor creates a new timer whose associated thread has the specified name.
4	Timer(String name, boolean isDaemon) This constructor creates a new timer whose associated thread has the specified name, and may be specified to run as a daemon.



Class methods

Sr.No.	Method & Description
1	<u>void cancel()</u> This method terminates this timer, discarding any currently scheduled tasks.

2	<u>int purge()</u> This method removes all cancelled tasks from this timer's task queue.
3	<u>void schedule(TimerTask task, Date time)</u> This method schedules the specified task for execution at the specified time.
4	<u>void scheduleAtFixedRate(TimerTask task, Date firstTime, long period)</u> This method schedules the specified task for repeated fixed-rate execution, beginning at the specified time.

Methods inherited

This class inherits methods from the following classes –

java.util.Object



Scheduling a Task to Execute using Timer Example

The following example shows the usage of Java Timer schedule(TimerTask, Date) method to schedule a timer operation. We've created a timer object using a CustomTimerTask object. CustomTimerTask is custom class extending TimerTask class and implements the run() method which will execute at scheduled time. Then we created a timer object and scheduled a task using schedule() to execute task now.

```
import java.util.Date;
import java.util.Timer;
import java.util.TimerTask;
```

```
public class TimerDemo {
    public static void main(String[] args) {

        // creating timer task, timer
        TimerTask tasknew = new CustomTimerTask();
        Timer timer = new Timer("test",true);

        // scheduling the task
        timer.schedule(tasknew, new Date());

        try {
            Thread.sleep(1000);
        } catch (InterruptedException e) {
            System.out.println("Error");
        }
    }
}
```

```
class CustomTimerTask extends TimerTask {

    @Override
    public void run() {
        System.out.println("working on");
    }
}
```

Output

Let us compile and run the above program, this will produce the following result.
working on

Vector Class:

Vector implements a dynamic array. It is similar to ArrayList, but with two differences –

- Vector is synchronized.
- Vector contains many legacy methods that are not part of the collections framework.

Vector proves to be very useful if you don't know the size of the array in advance or you just need one that can change sizes over the lifetime of a program.

Following is the list of constructors provided by the vector class.

Sr.No.	Constructor & Description
--------	---------------------------

1	Vector()
---	------------------

	This constructor creates a default vector, which has an initial size of 10.
--	---

2	Vector(int size)
---	-------------------------

	This constructor accepts an argument that equals to the required size, and creates a vector whose initial capacity is specified by size.
--	--

3	Vector(int size, int incr)
---	-----------------------------------

	This constructor creates a vector whose initial capacity is specified by size and whose increment is specified by incr. The increment specifies the number of elements to allocate each time that a vector is resized upward.
--	---

4	Vector(Collection c)
---	-----------------------------

	This constructor creates a vector that contains the elements of collection c.
--	---

Apart from the methods inherited from its parent classes, Vector defines the following methods –

Sr.No.	Method & Description
1	<u>boolean add(E e)</u> This method appends the specified element to the end of this Vector.
2	<u>boolean addAll(Collection<? extends E> c)</u> This method appends all of the elements in the specified Collection to the end of this Vector.
3	<u>void addElement(E obj)</u> This method adds the specified component to the end of this vector, increasing its size by one.
4	<u>int capacity()</u> This method returns the current capacity of this vector.
5	<u>void clear()</u> This method removes all of the elements from this vector.
6	<u>clone clone()</u> This method returns a clone of this vector.
7	<u>boolean contains(Object o)</u> This method returns true if this vector contains the specified element.

8	<u>boolean containsAll(Collection<?> c)</u> This method returns true if this Vector contains all of the elements in the specified Collection.
9	<u>void copyInto(Object[] anArray)</u> This method copies the components of this vector into the specified array.
10	<u>E elementAt(int index)</u> This method returns the component at the specified index.
11	<u>Enumeration<E> elements()</u> This method returns an enumeration of the components of this vector.
12	<u>void ensureCapacity(int minCapacity)</u> This method increases the capacity of this vector, if necessary, to ensure that it can hold at least the number of components specified by the minimum capacity argument.
13	<u>boolean equals(Object o)</u> This method compares the specified Object with this Vector for equality.
14	<u>E firstElement()</u> This method returns the first component (the item at index 0) of this vector.
15	<u>void forEach(Consumer<? super E> action)</u> This method performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
16	<u>E get(int index)</u> This method returns the element at the specified position in this Vector.
17	<u>int hashCode()</u> This method returns the hash code value for this Vector.
18	<u>int indexOf(Object o)</u> This method returns the index of the first occurrence of the specified element in this vector, or -1 if this vector does not contain the element.
19	<u>void insertElementAt(E obj, int index)</u> This method inserts the specified object as a component in this vector at the specified index.
20	<u>boolean isEmpty()</u> This method tests if this vector has no components.
21	<u>Iterator<E> iterator()</u> This method returns an iterator over the elements in this list in proper sequence.
22	<u>E lastElement()</u> This method returns the last component of the vector.
23	<u>int lastIndexOf(Object o)</u>

	This method returns the index of the last occurrence of the specified element in this vector, or -1 if this vector does not contain the element.
24	<u>ListIterator<E> listIterator()</u> This method returns a list iterator over the elements in this list (in proper sequence).
25	<u>E remove(int index)</u> This method removes the element at the specified position in this Vector.
26	<u>boolean removeAll(Collection<?> c)</u> This method removes from this Vector all of its elements that are contained in the specified Collection.
27	<u>void removeAllElements()</u> This method removes all components from this vector and sets its size to zero.
28	<u>boolean removeElement(Object obj)</u> This method removes the first occurrence of the argument from this vector.
29	<u>void removeElementAt(int index)</u> This method deletes the component at the specified index.
30	<u>boolean removeIf(Predicate<? super E> filter)</u> Removes all of the elements of this collection that satisfy the given predicate.
31	<u>boolean retainAll(Collection<?> c)</u> This method retains only the elements in this Vector that are contained in the specified Collection.
32	<u>E set(int index, E element)</u> This method replaces the element at the specified position in this Vector with the specified element.
33	<u>void setElementAt(E obj, int index)</u> This method sets the component at the specified index of this vector to be the specified object.
34	<u>void setSize(int newSize)</u> This method sets the size of this vector.
35	<u>int size()</u> This method returns the number of components in this vector.
36	<u>Spliterator<E> spliterator()</u> Creates a late-binding and fail-fast Spliterator over the elements in this list.
37	<u>List<E> subList(int fromIndex, int toIndex)</u> This method returns a view of the portion of this List between fromIndex, inclusive, and toIndex, exclusive.
38	<u>object[] toArray()</u>

	This method returns an array containing all of the elements in this Vector in the correct order.
39	<u>String toString()</u> This method returns a string representation of this Vector, containing the String representation of each element.
40	<u>void trimToSize()</u> This method trims the capacity of this vector to be the vector's current size.

```

1. import java.util.*;
2. public class VectorExample1 {
3.     public static void main(String args[]) {
4.         //Create an empty vector with initial capacity 4
5.         Vector<String> vec = new Vector<String>(4);
6.         //Adding elements to a vector
7.         vec.add("Tiger");
8.         vec.add("Lion");
9.         vec.add("Dog");
10.        vec.add("Elephant");
11.        //Check size and capacity
12.        System.out.println("Size is: "+vec.size());
13.        System.out.println("Default capacity is: "+vec.capacity());
14.        //Display Vector elements
15.        System.out.println("Vector element is: "+vec);
16.        vec.addElement("Rat");
17.        vec.addElement("Cat");
18.        vec.addElement("Deer");
19.        //Again check size and capacity after two insertions
20.        System.out.println("Size after addition: "+vec.size());
21.        System.out.println("Capacity after addition is: "+vec.capacity());
22.        //Display Vector elements again
23.        System.out.println("Elements are: "+vec);
24.        //Checking if Tiger is present or not in this vector
25.        if(vec.contains("Tiger"))
26.        {
27.            System.out.println("Tiger is present at the index " +vec.indexOf("Tiger"));
28.        }
29.        else
30.        {
31.            System.out.println("Tiger is not present in the list.");
32.        }
33.        //Get the first element
34.        System.out.println("The first animal of the vector is = "+vec.firstElement());
35.        //Get the last element
36.        System.out.println("The last animal of the vector is = "+vec.lastElement());
37.    }
38. }

```

Output:

- 39. Size is: 4
- 40. Default capacity is: 4
- 41. Vector element is: [Tiger, Lion, Dog, Elephant]
- 42. Size after addition: 7
- 43. Capacity after addition is: 8
- 44. Elements are: [Tiger, Lion, Dog, Elephant, Rat, Cat, Deer]
- 45. Tiger is present at the index 0
- 46. The first animal of the vector is = Tiger
- 47. The last animal of the vector is = Deer